Ashoka University
Artificial Intelligence: Assignment 2 (Coding Part)

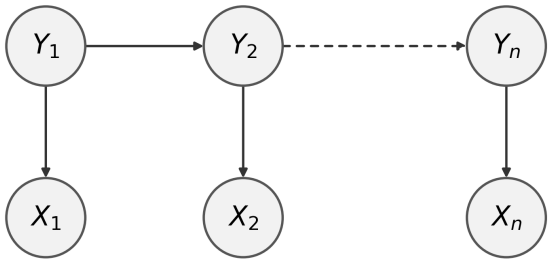
1 Hidden Markov Models

Aryan is a fervent ice cream enthusiast who indulges more as the temperature drops. Over n days we have a sequence of temperatures $Y_1, \ldots, Y_n$ and the corresponding ice cream consumption $X_1, \ldots, X_n$.

The daily ice cream intake X_t takes values 0, 1, or 2 servings. The temperature Y_t takes values 0 (cold), 1 (warm), or 2 (hot).

This is modeled as a Hidden Markov Model, where Y is the hidden state (temperature) and X is the observation (ice cream consumption). Two assumptions hold:

$$P(y_i \mid y_1, \ldots, y_{i-1}) = P(y_i \mid y_{i-1})$$
$$P(x_i \mid y_1, \ldots, y_i) = P(x_i \mid y_i)$$



The distribution $P(y_{i+1} \mid y_i)$ is the transition probability, $P(x_i \mid y_i)$ is the emission probability, and $P(y_1)$ is the initial state distribution.

An HMM class is provided in the Google Colab notebook:

`https://colab.research.google.com/drive/1wY6zP05UJJdKFnqJ05_YdOSDU5NF9HF2`

The class stores the initial state distribution, the transition probabilities and the emission probabilities. For instance, `transition_probability[y1][y2]` fetches $P(Y_{i+1} = y_2 \mid Y_i = y_1)$, so the first index is the current state and the second index is the next state. Each row sums to 1. Similarly, `emission_probability[y][x]` fetches $P(X_i = x \mid Y_i = y)$.

Suppose we observe the ice cream counts $x_1, \ldots, x_n$. The aim is to find the temperature sequence $y_1, \ldots, y_n$ that maximizes $P(y_1, \ldots, y_n \mid x_1, \ldots, x_n)$.

(a) For a chosen $x_1, \ldots, x_n$ and $y_1, \ldots, y_n$, how do you compute the joint distribution $P(y_1, \ldots, y_n, x_1, \ldots, x_n)$?

(b) Implement the function `joint_prob` to calculate $P(y_1, \ldots, y_n, x_1, \ldots, x_n)$. The input `outputs` is a list of the n observed ice cream consumptions and `hiddens` is a list of the hidden temperature states. Type annotations are only guidelines; Python does not check them.

(c) Complete `get_most_likely_hidden_states`, which takes an HMM object and the observed consumptions. For every possible temperature sequence (such as $[0, 0, 0, 0, 0]$, $[0, 0, 0, 0, 1]$, ...), compute $P(y_1, \ldots, y_n, x_1, \ldots, x_n)$. Then find

$$P(y_1, \ldots, y_n \mid x_1, \ldots, x_n) = \frac{P(y_1, \ldots, y_n, x_1, \ldots, x_n)}{P(x_1, \ldots, x_n)},$$

where $P(x_1, \ldots, x_n)$ is obtained by summing the joint over all possible $y_1, \ldots, y_n$. Sort by the conditional probability and output the 10 most likely sequences along with their joint probabilities. A Python list cannot be a dictionary key, so convert it to a tuple first with `tuple()`.

2 Variable Elimination

In this problem you will implement the variable elimination algorithm for exact inference in a Bayesian network. The starter code is provided in the Google Colab notebook:

`https://colab.research.google.com/drive/1QvN4yai9iQkXsDyepxWOPkKNp5xFidTK`

The notebook defines the `Variable`, `Factor` and `BN` classes, a sample network for testing, and a set of tests. You only fill in the five functions listed below.

Background. A factor is a function over a set of variables, called its scope. In the notebook, a factor stores a table with one value for each setting of the variables in its scope. Every conditional probability table of a Bayesian network is a factor. Variable elimination answers a query $P(Q \mid E)$ by applying three operations on factors, and then normalizing:

- restrict a factor to a given value of a variable (used for the evidence),
- multiply factors together,
- sum a variable out of a factor.

(a) Implement `normalize`. The input is a list of numbers. Return a new list of numbers, proportional to the input, that sums to 1.

(b) Implement `restrict_factor`. The input is a factor, a variable in its scope, and a value from the domain of that variable. Return a new factor that is the restriction of the input factor to that value. Do not modify the input factor.

(c) Implement `sum_out_variable`. The input is a factor and a variable in its scope. Return a new factor obtained by summing that variable out. Do not modify the input factor.

(d) Implement `multiply_factors`. The input is a list of factors. Return a new factor equal to their product, whose scope contains every variable that appears in any of the input factors. Do not modify the input factors.

(e) Implement `VE`. The input is a Bayes net, a query variable Q, and a list of evidence variables E, each of which already has its value set with `set_evidence`. Compute $P(Q = d \mid E)$ for every value d in the domain of Q, and return these probabilities as a list in the order of the domain. Do not modify any factor of the Bayes net.

Use only the Python standard library, and do not edit the classes or the sample network in the notebook. Run the test cells at the end of the notebook to check each function on small examples and to check `VE` on several queries of the sample network. Passing them is a good sign but not a proof, so write some tests of your own as well.

Submission

Submit the completed notebooks for both problems.